Automata on Graph Alphabets
Extended Abstract

Hugo Bazille[1] and Uli Fahrenberg[2]

[1] EPITA Research Lab (LRE), France
[2] LMF, Université Paris-Saclay

The theory of finite automata concerns itself with words over an alphabet Σ, that is, the free monoid on Σ together with concatenation and without further structure. There are, however, important applications which use alphabets which are structured in some sense, for example in database theory [6], model checking of systems with data [5,8], type checking [7], or concurrency theory [2].

Here we work with a particular type of structured data, namely an alphabet which is given as a (finite or infinite) directed graph. This constrains concatenation: two strings may only be concatenated if the end vertex of the first is equal to the start vertex of the second.

We give an example for motivation. In a concurrent setting, processes' behavior depends on whether or not they have access to shared resources. Figure 1 shows a simple graph alphabet designed to reflect this, where the action available depends on the state: in unsafe state, processes may do a, but after locking the resource (P), they can do b, until the resource is released (V). Note that the graph in Fig. 1 only constrains the *alphabet*; processes themselves may have arbitrarily complex behavior, but they are by design restricted to not do b while in unsafe state.

Another, more comprehensive example comes from higher-dimensional automata theory. We have in [1–3] introduced ST-automata: state-labeled automata on an alphabet of so-called starters and terminators. These are best understood as automata on an infinite graph alphabet where vertices denote events which are running and edges are starters and terminators.

A *graph alphabet* (V, Σ, d_0, d_1) consists of sets V and Σ together with source and target mappings $d_0, d_1 : \Sigma \to V$. We will generally omit d_0 and d_1 from the signature. Both V and Σ may be finite or infinite, as may the automata we define next.

Definition 1. *An* automaton *on graph alphabet* (V, Σ) *is a structure* $A = (Q, I, F, E, s, t, \mu, \lambda)$ *consisting of a set of states* Q *with initial and accepting states* $I, F \subseteq Q$*, a set of transitions* E *with source and target mappings* $s, t : E \to Q$*, and labelings* $\mu : Q \to V$*,* $\lambda : E \to \Sigma$*. We require that* $\mu(s(e)) = d_0(\lambda(e))$ *and* $\mu(t(e)) = d_1(\lambda(e))$ *for every* $e \in E$*.*

That is, the transitions of A are labeled with elements of Σ, but in a way consistent with the graph (V, Σ): a transition labeled a must emanate from a state labeled $d_0(a)$ and lead to a state labeled $d_1(a)$. We may understand the state labels as *types* which restrict the application of transitions.

A *path* in A is an alternating sequence $\pi = (q_0, e_1, q_1, \dots, e_n, q_n)$ of states and transitions such that $s(e_i) = q_{i-1}$ and $t(e_i) = q_i$ for all i; we naturally expand this notation to $s(\pi) = q_0$ and $t(\pi) = q_n$. The *label* of π is $\lambda(\pi) = (\mu(q_0), \lambda(e_1) \cdots \lambda(e_n), \mu(q_n))$.

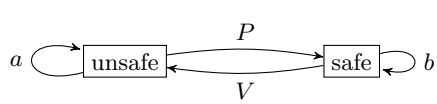

Figure 1: Alphabet for locking and releasing a resource.

We keep the labels of the start and target state of π in $\lambda(\pi)$ as we need typing information for concatenation. The shortest paths in A are the constant paths $\pi = (q)$ for $q \in Q$; the label of such a path π is $\lambda(\pi) = \mu(q)$. A path π is *accepting* if $s(\pi) \in I$ and $t(\pi) \in F$. The *language* of A is $L(A) = \{\lambda(\pi) \mid \pi \text{ accepting path in } A\}$.

We develop the beginnings of an automata theory for languages on graph alphabets. We show that they admit a Kleene theorem, relating rational and regular languages, and a Myhill-Nerode theorem, stating that languages are regular iff they have finite prefix or, equivalently, suffix quotient (subject to an easy condition on the graph). Further, automata on graph alphabets are determinizable and complementable, and minimal deterministic automata exist.

Finally, we also investigate a generalization of our setting to alphabets which form simplicial sets. One main key difference is that languages are no more freely generated. However, we conjecture that also in this setting, Kleene and Myhill-Nerode theorems hold.

This is an extended abstract of the preprint [4] with the same title.

References

[1] Amazigh Amrane, Hugo Bazille, Emily Clement, Uli Fahrenberg, and Krzysztof Ziemiański. Presenting interval pomsets with interfaces. In Uli Fahrenberg, Wesley Fussner, and Roland Glück, editors, *Relational and Algebraic Methods in Computer Science - 21st International Conference, RAMiCS 2024*, volume 14787 of *Lecture Notes in Computer Science*, pages 28–45. Springer, 2024.

[2] Amazigh Amrane, Hugo Bazille, Uli Fahrenberg, and Krzysztof Ziemiański. Closure and decision properties for higher-dimensional automata. *Theoretical Computer Science*, 1036:115156, 2025.

[3] Hugo Bazille, Jérémy Dubut, Uli Fahrenberg, and Krzysztof Ziemianski. Variants of higher-dimensional automata. *CoRR*, abs/2601.17537, 2026. https://arxiv.org/abs/2601-17537.

[4] Hugo Bazille and Uli Fahrenberg. Automata on graph alphabets. *CoRR*, abs/2602.10036, 2026. https://arxiv.org/abs/2602.10036.

[5] Stéphane Demri and Karin Quaas. Concrete domains in logics: a survey. *ACM SIGLOG News*, 8(3):6–29, 2021.

[6] Frank Neven, Thomas Schwentick, and Dan Suciu, editors. *Foundations of Semistructured Data, 6.-11. February 2005*, volume 05061 of *Dagstuhl Seminar Proceedings*. IBFI, Schloss Dagstuhl, Germany, 2005.

[7] Jim E. Newton. Type-checking heterogeneous sequences in a simple embeddable type system. In Esra Erdem and Germán Vidal, editors, *Practical Aspects of Declarative Languages - 27th International Symposium, PADL 2025*, volume 15537 of *Lecture Notes in Computer Science*, pages 18–34. Springer, 2025.

[8] Dominik Peteler and Karin Quaas. Deciding emptiness for constraint automata on strings with the prefix and suffix order. In Stefan Szeider, Robert Ganian, and Alexandra Silva, editors, *47th International Symposium on Mathematical Foundations of Computer Science, MFCS 2022*, volume 241 of *LIPIcs*, pages 76:1–76:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.